Day 2: Kubernetes Core Concepts – Pods, Deployments, Services, Ingress

Welcome to Day 2 of the Containers & Kubernetes workshop! Yesterday you mastered Docker – building images, running containers, and managing multi-container applications. Today, we step into the world of **orchestration** with Kubernetes (K8s). You will learn:

- Why we need orchestration and what Kubernetes offers.
- The architecture of a Kubernetes cluster (control plane vs worker nodes).
- The fundamental building block: **Pods**.
- How to describe Kubernetes resources using YAML.
- How to run and manage applications using **Deployments**.
- How to expose applications internally and externally using **Services**.
- How to route HTTP traffic with **Ingress**.

By the end of the day, you'll have a solid understanding of core Kubernetes objects and be able to deploy a scalable web application on your local Minikube cluster.

Prerequisites

- Docker installed and working (from Day 1).
- Minikube installed and started (`minikube start`).
- `kubectl` installed and configured.
- Verify your environment:

```
minikube status
kubectl cluster-info
kubectl get nodes
```

09:00 – 10:30 – Conceptual Session: Kubernetes Architecture

1.1 Why Orchestration?

Problem with managing containers manually

Running containers with Docker on a single host is fine for development. But in production, you have many containers across many hosts. You need to:

- **Schedule** containers to suitable nodes.
- **Maintain desired state** (e.g., always 3 replicas).
- **Self-heal** when containers fail.
- **Scale** up/down based on load.
- **Roll out updates** without downtime.
- **Manage networking** across hosts.

- **Store and manage secrets** and configuration.

This is where **container orchestration** comes in. Kubernetes is the leading orchestration platform.

Analogy

Think of an orchestra conductor. Each musician (container) plays their part, but the conductor ensures they all play together, start/stop at the right time, and adapt to changes. Kubernetes is the conductor for your containers.

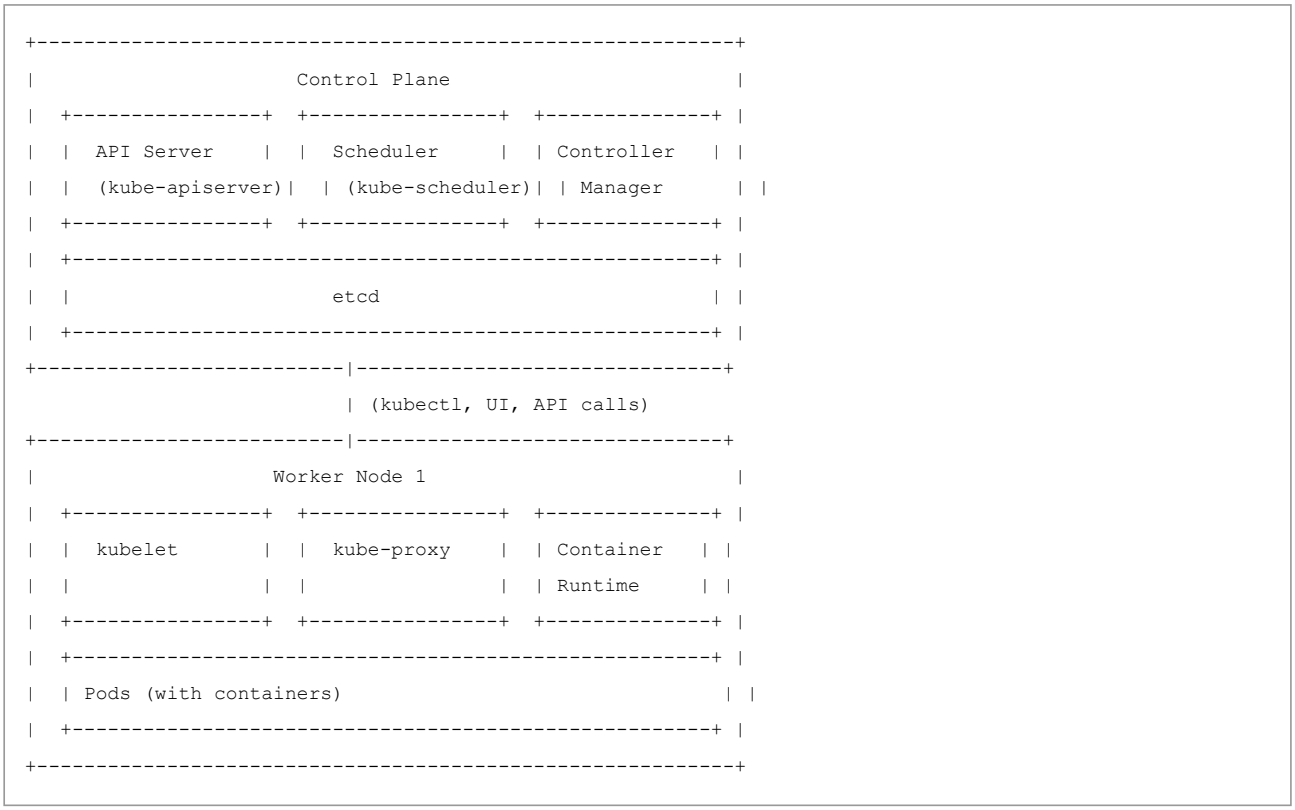
Desired State Management

Kubernetes operates on a **declarative model**: you tell it what you want (e.g., “run 3 copies of my web app”), and Kubernetes continuously works to make the current state match that desired state. This is called the **reconciliation loop**.

1.2 Kubernetes Architecture

A Kubernetes cluster consists of two main parts: the **Control Plane** (master) and the **Worker Nodes**.

Diagram: High-level Architecture



Control Plane Components

- **API Server (`kube-apiserver`)**: The front door of the cluster. All administrative tasks (via `kubectl`, UI, or API) go through the API server. It validates and processes requests, then updates the corresponding objects in etcd.
- **etcd**: A distributed key-value store that holds the entire cluster state (configuration, desired state, current state). It’s the source of truth.
- **Scheduler (`kube-scheduler`)**: Watches for newly created Pods that are not yet assigned to a node, and selects a suitable node for them based on resource requirements, policies, etc.
- **Controller Manager (`kube-controller-manager`)**: Runs controller processes that regulate the cluster. For example:
  - **Node Controller**: Monitors node health.
  - **Replication Controller**: Ensures the correct number of Pod replicas are running.
  - **Endpoint Controller**: Populates Endpoints objects (joins Services & Pods).
  - ... and many more.

Worker Node Components

- **kubelet**: An agent running on each node. It communicates with the API server, ensures containers are running in Pods as expected, and reports node and Pod status.
- **kube-proxy**: A network proxy that maintains network rules on nodes. It enables communication to your Pods from inside or outside the cluster.
- **Container Runtime**: The software that runs containers (e.g., Docker, containerd, CRI-O). Kubernetes uses the Container Runtime Interface (CRI) to interact with the runtime.

1.3 Pods – The Atomic Unit

A **Pod** is the smallest and simplest Kubernetes object. It represents a single instance of a running process in your cluster.

What's inside a Pod?

- One or more containers (usually one main container, plus optional helper containers called **sidecars**).
- Shared storage (volumes).
- Unique IP address per Pod.
- Shared network namespace: containers in the same Pod can communicate via `localhost`.

Pod Lifecycle

- **Pending**: Pod is accepted but not yet running (e.g., pulling images).
- **Running**: Pod bound to a node, all containers created, at least one still running.
- **Succeeded**: All containers terminated successfully.
- **Failed**: All containers terminated, at least one failure.
- **CrashLoopBackOff**: Container repeatedly crashes.
- **Unknown**: State unknown (communication loss).

Diagram: Pod with One Container

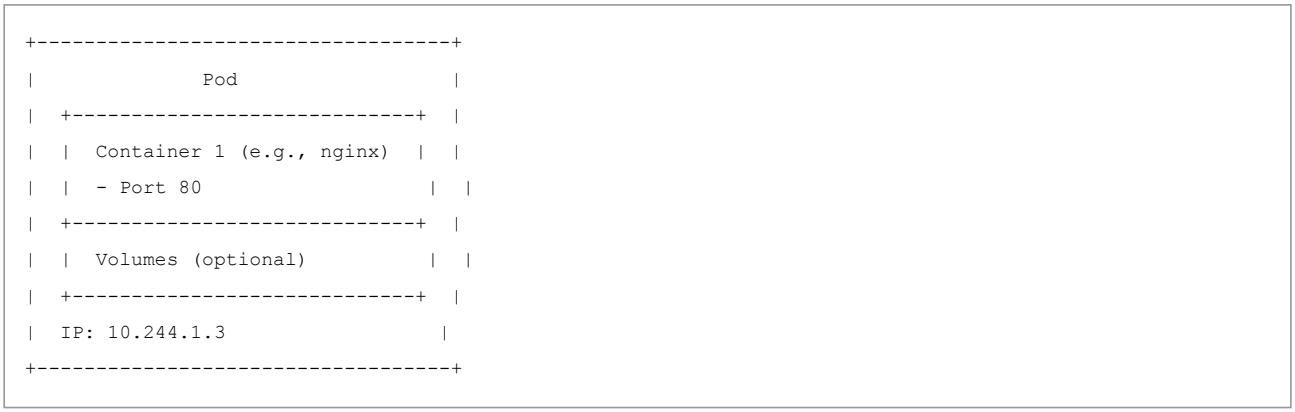
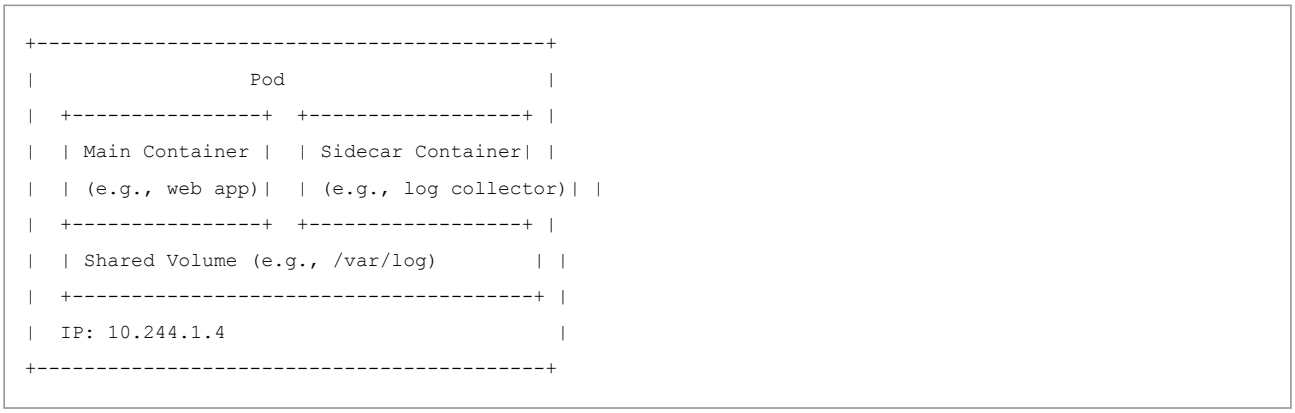


Diagram: Multi-container Pod (Sidecar Pattern)



Pods are **ephemeral** – they can be created, destroyed, and recreated. Their IP addresses change. That's why we need higher-level abstractions like **Deployments** and **Services**.

1.4 YAML Basics for Kubernetes

Kubernetes resources are defined in YAML (or JSON) files. A typical resource manifest has four top-level fields:

```
apiVersion: v1                 # Which version of the Kubernetes API to use
kind: Pod                       # The type of resource
metadata:                       # Metadata about the resource (name, namespace, labels)
  name: my-pod
  labels:
    app: myapp
spec:                           # The desired state (what you want)
  containers:
  - name: nginx
    image: nginx:latest
```

Common `kind` values

- `Pod`
- `Deployment`
- `Service`
- `Ingress`
- `ConfigMap`
- `Secret`
- `Namespace`

Common `apiVersion` values

- `v1` – for core objects like Pod, Service, ConfigMap, Secret.
- `apps/v1` – for Deployments, ReplicaSets, StatefulSets.
- `networking.k8s.io/v1` – for Ingress.

Important fields in `spec`

- `containers`: list of containers. Each has `name`, `image`, `ports`, `env`, `resources`, `volumeMounts`, etc.
- `volumes`: list of volumes that can be mounted by containers.
- `nodeSelector`, `affinity`: for scheduling constraints.
- `restartPolicy`: Always, OnFailure, Never.

Indentation matters! YAML is whitespace-sensitive. Use spaces, not tabs.

Demo: Writing a simple Pod YAML

We'll create a pod that runs nginx.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
```

10:45 – 12:30 – Hands-on Lab 1: Pods and Deployments

Exercise 1.1 – Start Minikube

If not already running, start Minikube with the Docker driver (or your preferred driver):

```
minikube start --driver=docker
```

Verify the cluster is healthy:

```
kubectl get nodes
kubectl cluster-info
```

Exercise 1.2 – Create Pods Imperatively

The quickest way to create a Pod is using `kubectl run` (imperative command):

```
kubectl run nginx-pod --image=nginx --restart=Never
```

- `--restart=Never` tells Kubernetes to create a Pod (not a Deployment). Wait, actually `kubectl run` can create different resources; we use `--restart=Never` to ensure it's a Pod.

Check Pod status:

```
kubectl get pods
kubectl describe pod nginx-pod
```

View logs:

```
kubectl logs nginx-pod
```

Execute a command inside the Pod (e.g., open a shell):

```
kubectl exec -it nginx-pod -- /bin/bash
# Inside: ls, exit
```

Delete the Pod:

```
kubectl delete pod nginx-pod
```

Exercise 1.3 – Create Pod Declaratively (YAML)

1. Create a file named `pod.yaml` with the content from the demo above.
2. Apply it:

```
kubectl apply -f pod.yaml
```

3. Verify:

```
kubectl get pods
kubectl get pod nginx-pod -o wide   # shows IP and node
```

4. Test access (we'll need a Service later, but you can use port-forwarding):

```
kubectl port-forward pod/nginx-pod 8080:80
```

In another terminal, `curl http://localhost:8080`. Stop port-forward with Ctrl+C.

5. Delete the Pod:

```
kubectl delete -f pod.yaml
```

Exercise 1.4 – Create a Deployment

A **Deployment** manages a set of identical Pods (a ReplicaSet) and provides rolling updates, scaling, and self-healing.

Create `deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```

Apply it:

```
kubectl apply -f deployment.yaml
```

Check the status:

```
kubectl get deployments
kubectl get pods -w   # watch pods being created
```

List all resources created:

```
kubectl get all
```

You'll see the Deployment, a ReplicaSet, and three Pods.

Exercise 1.5 – Rolling Update and Rollback

Update the nginx image to a newer version (e.g., `1.25`):

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.25
```

Alternatively, edit the deployment directly:

```
kubectl edit deployment nginx-deployment
```

Watch the rolling update:

```
kubectl rollout status deployment/nginx-deployment
```

View rollout history:

```
kubectl rollout history deployment/nginx-deployment
```

If something goes wrong, rollback to the previous revision:

```
kubectl rollout undo deployment/nginx-deployment
```

Check that the image reverted.

Exercise 1.6 – Scale the Deployment

Scale up to 5 replicas:

```
kubectl scale deployment nginx-deployment --replicas=5
```

Verify:

```
kubectl get pods
```

Scale back to 3.

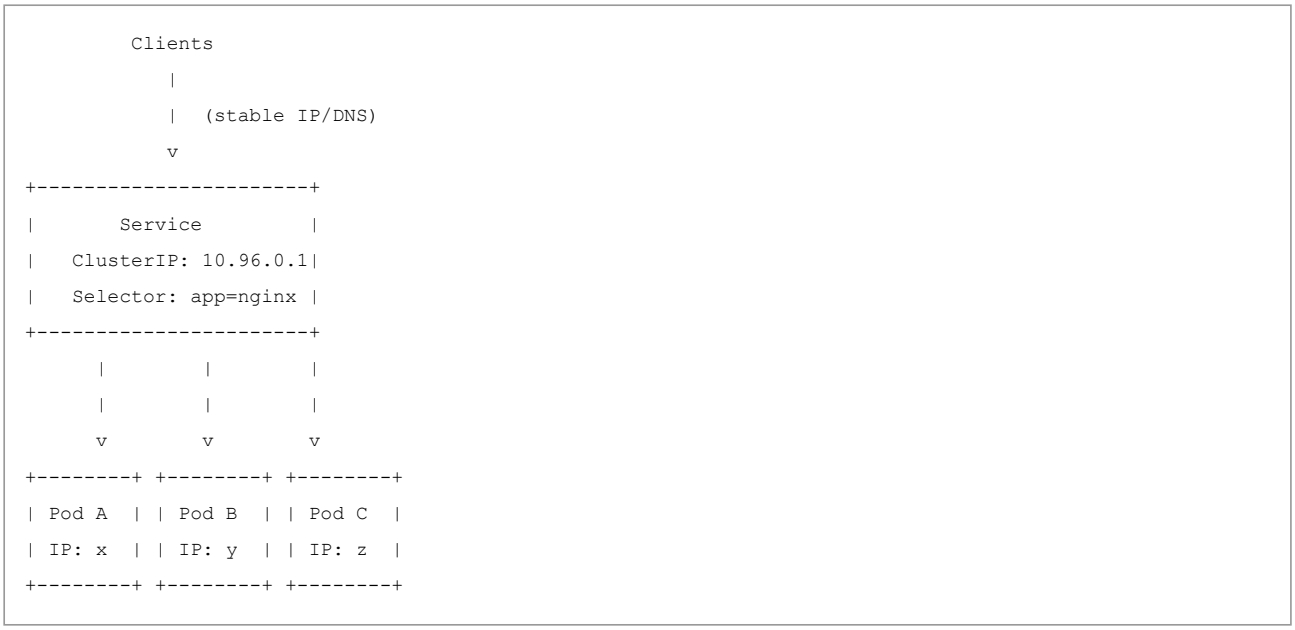
13:30 – 15:00 – Conceptual Session: Services & Ingress

2.1 Why Services?

Pods are ephemeral – they come and go, get new IPs, and can be deleted by scaling events. If one part of your application needs to talk to another (e.g., a frontend talking to a backend), you cannot rely on Pod IPs directly.

Services provide a stable abstraction. They group a set of Pods (using label selectors) and provide a single, stable IP and DNS name. They also do basic load balancing across the Pods.

Diagram: Service Abstraction

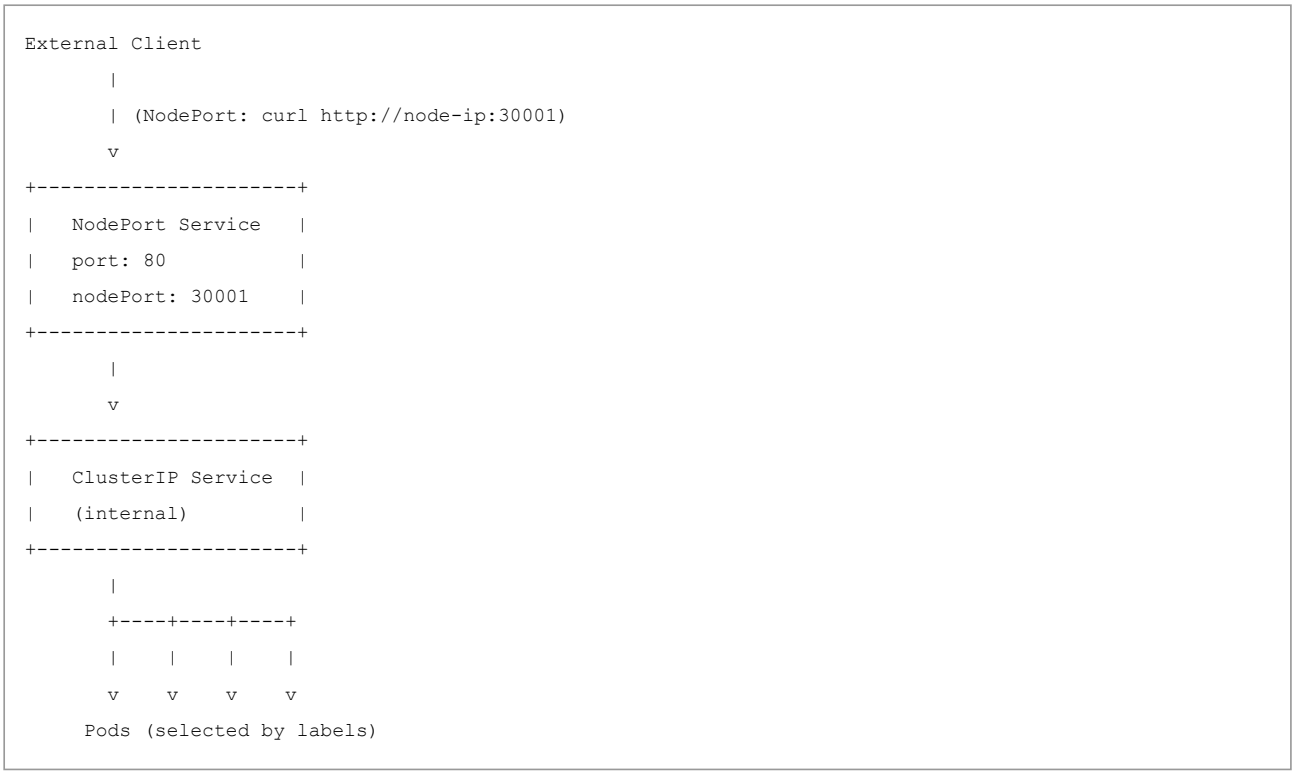


2.2 Service Types

Kubernetes supports several types of Services, depending on how you want to expose them:

Type	Description	Typical Use Case
ClusterIP	Exposes the Service on a cluster-internal IP. Only reachable from within the cluster.	Internal communication (backend to backend).
NodePort	Exposes the Service on each Node's IP at a static port (the NodePort). You can reach the Service from outside the cluster by `<NodeIP>:<NodePort>`.	Development, simple external access.
LoadBalancer	Exposes the Service externally using a cloud provider's load balancer. NodePort and ClusterIP are created automatically.	Production, cloud environments.
ExternalName	Maps the Service to an external DNS name (no proxying).	Integrating external services.

Diagram: Service Types

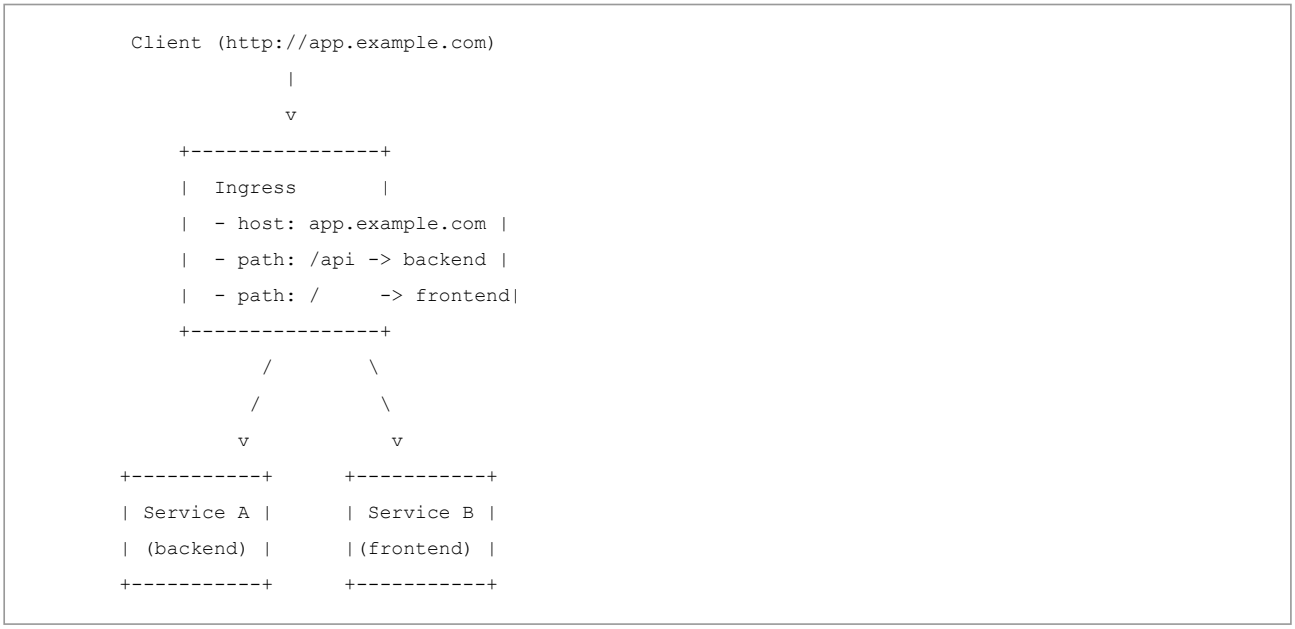


2.3 Ingress

Ingress manages external access to HTTP/HTTPS services, providing features like host-based routing, path-based routing, TLS termination, and more. Ingress is not a Service; it's a separate resource that relies on an **Ingress Controller** (e.g., NGINX Ingress Controller, Traefik) to function.

Without Ingress, you would need a separate LoadBalancer Service for every app. Ingress allows you to consolidate routing rules.

Diagram: Ingress Routing



Ingress Example YAML:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example-ingress
spec:
  ingressClassName: nginx   # specify controller
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: myapp-service
            port:
              number: 80
```

2.4 Service Discovery

Kubernetes provides two ways for Pods to discover Services:

1. **Environment variables**: When a Pod starts, kubelet adds environment variables for each active Service (e.g., `MYAPP_SERVICE_HOST`, `MYAPP_SERVICE_PORT`). This is simple but only includes Services that exist before the Pod starts.
2. **DNS**: Kubernetes runs a DNS server (CoreDNS) inside the cluster. Every Service gets a DNS name: `<service-name>.<namespace>.svc.cluster.local`. Pods can use that name to reach the Service.

Example: A Pod in the `default` namespace can reach a Service named `my-service` via `my-service` (short name) or `my-service.default.svc.cluster.local` (FQDN).

15:15 – 16:45 – Hands-on Lab 2: Services & Ingress

Exercise 2.1 – Expose Deployment with ClusterIP

First, ensure your nginx deployment is running with 3 replicas.

Create a ClusterIP Service for the deployment:

```
kubectl expose deployment nginx-deployment --port=80 --target-port=80 --name=nginx-service
```

Check the Service:

```
kubectl get svc
kubectl describe svc nginx-service
```

Note the ClusterIP (e.g., 10.96.123.45). To test from within the cluster, run a temporary Pod:

```
kubectl run test-pod --image=busybox --rm -it --restart=Never -- wget -O- nginx-service
```

You should see the Nginx welcome HTML.

Exercise 2.2 – NodePort Service

Create a NodePort Service for the same deployment:

```
kubectl expose deployment nginx-deployment --port=80 --target-port=80 --type=NodePort --name=nginx-nodeport
```

List Services:

```
kubectl get svc
```

You'll see `nginx-nodeport` with a ClusterIP and a NodePort (e.g., 30001). Get your Minikube node IP:

```
minikube ip
```

Now access from your host using `http://<minikube-ip>:<NodePort>`.

Minikube also provides a convenient command to open the service in a browser:

```
minikube service nginx-nodeport
```

Exercise 2.3 – Install NGINX Ingress Controller

Minikube comes with an addon for Ingress. Enable it:

```
minikube addons enable ingress
```

Wait for the ingress controller pod to be ready:

```
kubectl get pods -n ingress-nginx -w
```

(If you prefer to install manually, you can use the manifest from the plan, but the addon is simpler.)

Exercise 2.4 – Create Ingress Rule

Create a file `ingress.yaml`:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nginx-ingress
spec:
  ingressClassName: nginx
  rules:
  - host: nginx.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: nginx-service   # the ClusterIP service we created earlier
            port:
              number: 80
```

Apply it:

```
kubectl apply -f ingress.yaml
```

Now you need to make the hostname `nginx.local` resolve to your Minikube IP. Get the IP:

```
minikube ip
```

Edit your `/etc/hosts` file (admin privileges required) and add a line:

```
<minikube-ip>   nginx.local
```

On Linux/Mac, you can do:

```
echo "$(minikube ip) nginx.local" | sudo tee -a /etc/hosts
```

Now test with curl:

```
curl http://nginx.local
```

You should see the Nginx welcome page.

Exercise 2.5 – Cleanup

Delete the Ingress, Services, and Deployment:

```
kubectl delete ingress nginx-ingress
kubectl delete svc nginx-service nginx-nodeport
kubectl delete deployment nginx-deployment
```

Optionally, remove the hosts entry you added.

Daily Recap & Troubleshooting

Key Takeaways

- **Kubernetes architecture**: Control plane (API server, etcd, scheduler, controller manager) and worker nodes (kubelet, kube-proxy, container runtime).
- **Pod**: The smallest deployable unit; runs one or more containers.
- **Deployment**: Manages stateless applications, supports scaling, rolling updates, and rollbacks.
- **Service**: Provides stable IP and DNS for a set of Pods, with load balancing. Types: ClusterIP, NodePort, LoadBalancer.
- **Ingress**: Manages external HTTP(S) access with routing rules; requires an Ingress Controller.
- **YAML**: Used to declare desired state; fields: apiVersion, kind, metadata, spec.

Common Issues & Solutions

Problem	Likely Cause	Solution
`kubectl get nodes` stuck or no nodes	Minikube not running or not started properly.	Run `minikube start` and check status.
Pod stuck in `Pending`	Insufficient resources, or node not ready.	`kubectl describe pod` to see events.
Pod stuck in `ContainerCreating`	Image pull issue, or volume mount problem.	Check image name, network connectivity.
Service not accessible via NodePort	NodePort not exposed, firewall blocking.	Use `minikube service <name>` to open in browser.
Ingress not working	Ingress controller not running, or hosts entry missing.	Verify ingress controller pods; check `/etc/hosts`.
`curl: (7) Failed to connect`	Service not running, or wrong port.	Verify endpoints: `kubectl get endpoints`.
`kubectl exec` fails with “unable to upgrade connection”	Network issue, or container not ready.	Ensure Pod is `Running`.

Useful Commands Recap

Command	Description
`kubectl get pods`	List all Pods.
`kubectl describe pod <name>`	Show detailed Pod info and events.
`kubectl logs <pod>`	Fetch logs.
`kubectl exec -it <pod> -- <cmd>`	Execute command inside Pod.
`kubectl apply -f <file>`	Create/update resources from YAML.
`kubectl delete -f <file>`	Delete resources from YAML.
`kubectl get deployments`	List Deployments.
`kubectl scale deployment <name> --replicas=N`	Scale a Deployment.
`kubectl rollout status deployment/<name>`	Check rollout status.
`kubectl rollout undo deployment/<name>`	Rollback to previous version.
`kubectl expose deployment <name> ...`	Create a Service from a Deployment.
`kubectl get svc`	List Services.
`kubectl port-forward pod/<name> host:pod`	Forward a local port to a Pod.

Connection to Previous Modules

- **Scripting (Days 36–40)**: You used `kubectl` commands in the terminal; these can be automated in scripts.
- **Infrastructure (Days 45–47)**: Networking concepts (IP, DNS, load balancing) directly apply to Services and Ingress.

- **Security (Days 49–50)**: RBAC (covered Day 55) will control access to these resources; IAM principles map to Kubernetes authentication/authorisation.

Congratulations! You have successfully navigated the core concepts of Kubernetes. You now know how to deploy and expose applications.

Tomorrow we will dive into Helm and Kubernetes security (RBAC). Keep your Minikube cluster ready!